



APLIKASI PEMESANAN MAKANAN SEDERHANA



PRESENTED BY GROUP 2



ANGGOTA KELOMPOK



**Alya Naditha
Elviana Vania
Diva Septiyanti**



FUNGSI UTAMA SISTEM

- Menampilkan daftar menu dan harga— Sistem menampilkan pilihan makanan dan minuman beserta harganya agar pengguna dapat melihat opsi yang tersedia dengan cepat.
- Pemilihan menu, jumlah, dan keranjang— Pengguna dapat memilih menu, menentukan jumlah pesanan, lalu menambahkannya ke keranjang.
- Menghitung subtotal dan total pesanan— Sistem otomatis menghitung subtotal tiap item dan total akhir untuk mengurangi kesalahan perhitungan manual.
- Proses pembayaran— Sistem menyediakan metode pembayaran cash atau transfer agar transaksi lebih fleksibel dan praktis.
- Mencetak ringkasan transaksi— Sistem menghasilkan ringkasan pesanan, total pembayaran, dan metode pembayaran sebagai bukti transaksi.

ANALISIS KEBUTUHAN — NON-FUNGSIONAL

1

Usability

Tampilan dibuat sederhana, intuitif, dan mudah dipahami oleh kasir maupun pengguna baru.
Contoh: menu dapat dipilih dalam beberapa klik, label tombol jelas, dan alur pembayaran tidak membingungkan.

2

Performance

Proses subtotal, total pesanan, dan transaksi harus cepat agar antrian tidak menumpuk.
Contoh: perubahan jumlah menu langsung memperbarui total secara real-time tanpa memuat ulang halaman.

3

Reliability

Sistem harus menghasilkan perhitungan yang akurat dan konsisten pada setiap transaksi.
Contoh: total akhir harus sesuai dengan subtotal, pajak, dan metode pembayaran yang dipilih.

4

Maintainability

Kode dibuat dengan struktur OOP yang rapi agar mudah dikembangkan, diuji, dan diperbaiki.
Contoh: class menu, keranjang, transaksi, dan pembayaran dipisahkan agar fitur baru mudah ditambahkan.



PERANCANGAN SISTEM — UML

1

Class Diagram

Menunjukkan struktur kelas, atribut, metode, dan relasi antar objek seperti Menu , Pesanan , Pelanggan , dan Transaksi . Diagram ini membantu menjaga desain OOP tetap rapi.

2

Use Case Diagram

Menggambarkan interaksi pengguna dengan sistem, misalnya pelanggan melihat menu, membuat pesanan, membayar, dan admin mengelola data. Diagram ini memperjelas fitur utama aplikasi.

3

Activity Diagram

Memodelkan alur kerja proses, dari memilih makanan hingga konfirmasi pembayaran. Diagram ini membantu melihat urutan langkah dan potensi hambatan.

4

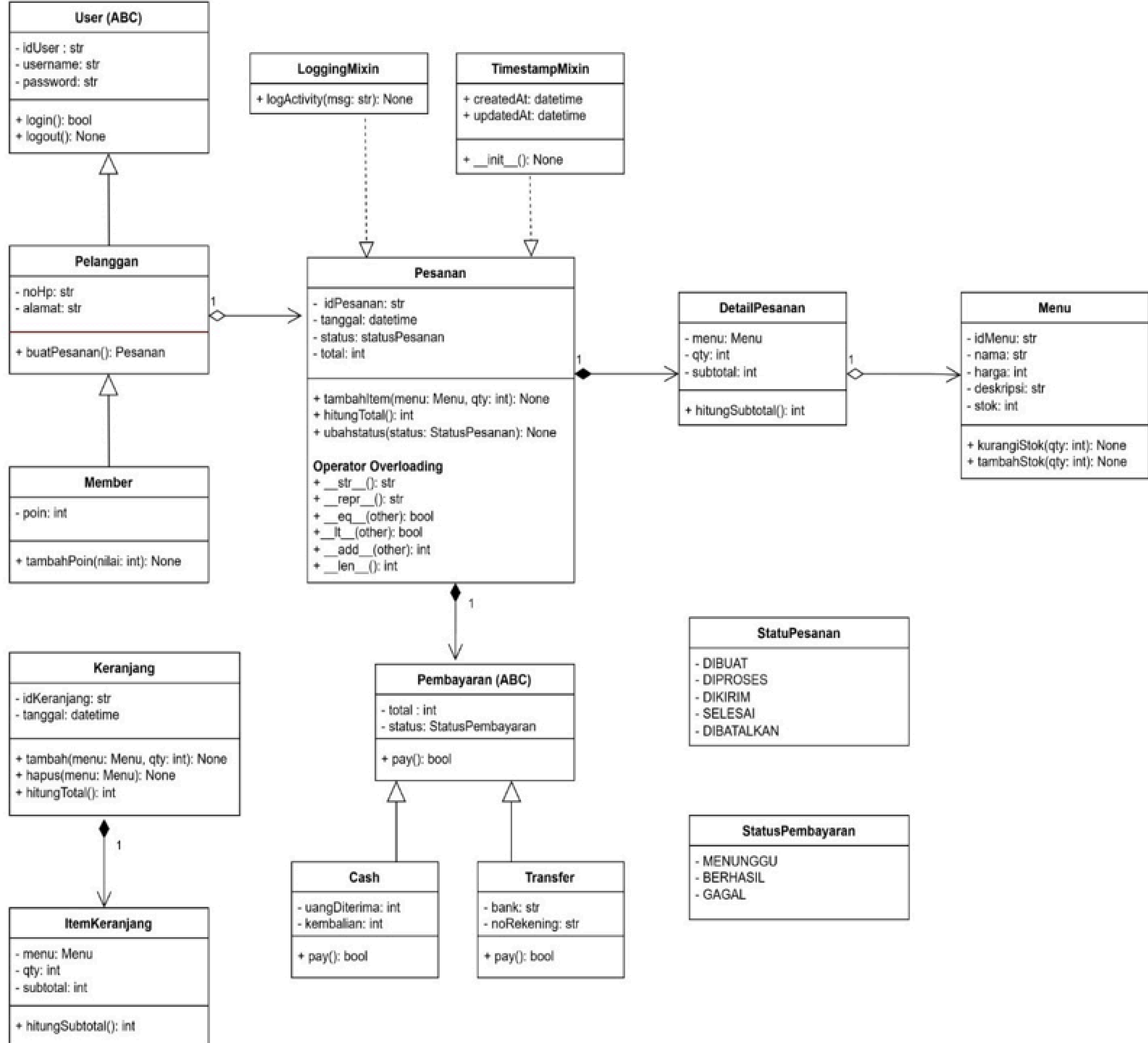
Sequence Diagram

Menjelaskan urutan interaksi antar objek dalam satu skenario, seperti saat pelanggan mengirim pesan lalu sistem memproses dan menyimpan data. Diagram ini memastikan komunikasi berjalan benar.

5

State Machine Diagram

Memperlihatkan perubahan status pesanan seperti dibuat, diproses, dibayar, disiapkan, dan selesai. Diagram ini menjaga perpindahan status tetap jelas dan konsisten.



CLASS DIAGRAM — RINGKASAN KELAS

- **User /Pelanggan /Member**

User adalah kelas induk untuk identitas dan autentikasi. Pelanggan mewarisi User untuk data profil dan riwayat transaksi, sedangkan Member menambahkan fitur loyalitas. Atribut penting: id, username, password, nama, email, statusAkun, poinLoyalitas, dan levelMember.

Method utama:

login(), logout(), ubahProfil(), tambahPoin(), dan cekStatusMember().

CLASS DIAGRAM — RINGKASAN KELAS

- **Menu**

Menu merepresentasikan item yang dapat dipesan dan menjadi data utama katalog. Atribut penting: idMenu, namaMenu, kategori, harga, stok, dan statusTersedia.

Method utama: kurangiStok(), tambahStok(), ubahHarga(), dan cekKetersediaan().

CLASS DIAGRAM — RINGKASAN KELAS

- **Keranjang & ItemKeranjang**

Keranjang menyimpan pilihan pelanggan sebelum pesanan dibuat, sedangkan ItemKeranjang mewakili detail tiap menu. Atribut penting: idKeranjang, idPelanggan, daftarItem, totalHarga, dan statusKeranjang.

Method utama: tambahItem(), hapusItem(), kosongkanKeranjang(), hitungTotal(), dan ubahJumlah().

CLASS DIAGRAM — RINGKASAN KELAS

- **Pemesanan & Pembayaran**

Pemesanan mengubah isi keranjang menjadi transaksi resmi, sementara Pembayaran menangani pelunasan. Atribut penting: idPesanan, nomorPesanan, totalAkhir, statusPesanan, metodePembayaran, nominalBayar, dan statusPembayaran. Method utama: buatPesanan(), ubahStatus(), prosesPembayaran(), validasiNominal(), dan cetakStruk().

IMPLEMENTASI KONSEP OBJECT-ORIENTED PROGRAMMING (OOP)

Encapsulation atau enkapsulasi merupakan konsep yang digunakan untuk menggabungkan data dan fungsi yang berkaitan ke dalam satu kesatuan yang disebut kelas (class). Penerapan enkapsulasi terlihat pada kelas pesanan yang bertanggung jawab mengelola data transaksi pelanggan. Perhitungan total pesanan dilakukan melalui method `hitungTotal()` sehingga proses perhitungan tetap terkontrol dan tidak bergantung pada manipulasi data secara langsung dari luar objek.

IMPLEMENTASI KONSEP OBJECT-ORIENTED PROGRAMMING (OOP)

Inheritance atau pewarisan merupakan mekanisme yang memungkinkan suatu kelas mewarisi atribut dan method dari kelas lain. Konsep ini bertujuan untuk mengurangi pengulangan kode serta meningkatkan efisiensi dalam pengembangan perangkat lunak. Pada aplikasi pemesanan makanan sederhana, pewarisan diterapkan pada Kelas User, Pelanggan, Member, dan Sistem Pembayaran.

User → Pelanggan → Member

IMPLEMENTASI KONSEP OBJECT-ORIENTED PROGRAMMING (OOP)

Polymorphism atau polimorfisme merupakan kemampuan objek untuk memiliki banyak bentuk perilaku melalui penggunaan method yang sama tetapi menghasilkan implementasi yang berbeda. Konsep ini memungkinkan sistem menjadi lebih fleksibel dalam menangani berbagai jenis objek.

Pada aplikasi pemesanan makanan sederhana, polimorfisme diterapkan pada proses pembayaran.

IMPLEMENTASI KONSEP OBJECT-ORIENTED PROGRAMMING (OOP)

Abstraction (Abstraksi) adalah konsep OOP yang digunakan untuk menyederhanakan sistem dengan menampilkan fitur penting dan menyembunyikan detail implementasi. Pada aplikasi pemesanan makanan sederhana, abstraksi diterapkan melalui penggunaan **Abstract Base Class (ABC)** pada kelas **User** dan **Pembayaran** sebagai kelas dasar yang tidak dapat dibuat objek secara langsung. Dengan demikian, pengguna hanya berinteraksi dengan fungsi yang diperlukan tanpa perlu mengetahui proses internalnya.

IMPLEMENTASI KONSEP OBJECT-ORIENTED PROGRAMMING (OOP)

Penerapan dan Evaluasi Prinsip SOLID membantu membuat aplikasi pemesanan makanan lebih terstruktur, mudah dipelihara, dan mudah dikembangkan. Setiap kelas memiliki tugas yang jelas, penggunaan pewarisan memudahkan penambahan fitur baru, serta sistem dibuat lebih fleksibel melalui kelas abstrak. Dengan demikian, aplikasi menjadi lebih modular dan mudah dikembangkan di masa depan.

IMPLEMENTASI DAN HASIL

Implementasi Struktur Kelas

Struktur kelas pada aplikasi dibangun berdasarkan rancangan UML yang telah dibuat. Setiap kelas memiliki tugas yang berbeda, seperti pengelolaan pengguna, menu makanan, keranjang, pesanan, dan pembayaran. Hubungan antar kelas menerapkan konsep pewarisan, komposisi, agregasi, dan abstraksi untuk menghasilkan sistem yang lebih terstruktur.

IMPLEMENTASI DAN HASIL

Implementasi Fitur Utama

Fitur utama yang diimplementasikan meliputi proses login, pengelolaan menu makanan, penambahan item ke keranjang, pembuatan pesanan, pemilihan metode pembayaran, serta pengelolaan status pesanan. Seluruh fitur dirancang agar dapat mendukung proses pemesanan makanan secara sederhana dan mudah digunakan.

IMPLEMENTASI DAN HASIL

Implementasi State Machine

State Machine diterapkan untuk mengatur perubahan status pesanan secara berurutan, mulai dari Dibuat, Diproses, Dikemas, Dikirim, hingga Selesai. Selain itu, sistem juga menyediakan status Dibatalkan untuk menangani kondisi tertentu. Pada proses pembayaran digunakan status Menunggu, Berhasil, dan Gagal.

IMPLEMENTASI DAN HASIL

Hasil dan Analisis Hasil

Pengujian menunjukkan bahwa seluruh fitur utama aplikasi, seperti login, keranjang, pemesanan, pembayaran, dan status pesanan, berjalan dengan baik. Penerapan OOP, UML, dan prinsip SOLID juga membuat sistem lebih terstruktur, mudah dipahami, dan mudah dikembangkan.

KESIMPULAN

Aplikasi pemesanan makanan berhasil dirancang menggunakan berbagai diagram UML, diimplementasikan dengan konsep OOP, serta diuji sehingga seluruh fitur berjalan sesuai kebutuhan. Selain itu, penerapan prinsip SOLID menghasilkan program yang lebih modular, mudah dipelihara, dan mudah dikembangkan di masa mendatang.



THANK YOU

PRESENTED BY GROUP 2